



What is Amazon
Relational Database
Service (Amazon RDS)?

Overview of Amazon RDS

Why do you want a managed relational database service? Because Amazon RDS takes over many of the difficult and tedious management tasks of a relational database:

- When you buy a server, you get CPU, memory, storage, and IOPS, all bundled together. With Amazon RDS, these are split apart so that you can scale them independently. If you need more CPU, less IOPS, or more storage, you can easily allocate them.
- Amazon RDS manages backups, software patching, automatic failure detection, and recovery.
- To deliver a managed service experience, Amazon RDS doesn't provide shell access to DB instances. It also restricts access to certain system procedures and tables that require advanced privileges.
- You can have automated backups performed when you need them, or manually create your own backup snapshot. You can use these backups to restore a database. The Amazon RDS restore process works reliably and efficiently.

Overview of Amazon RDS

- You can use the database products you are already familiar with: MySQL, MariaDB, PostgreSQL, Oracle, Microsoft SQL Server.
- You can get high availability with a primary instance and a synchronous secondary instance that you can fail over to when problems occur. You can also use MariaDB, Microsoft SQL Server, MySQL, Oracle, and PostgreSQL read replicas to increase read scaling.
- In addition to the security in your database package, you can help control who can access your RDS databases by using AWS Identity and Access Management (IAM) to define users and permissions. You can also help protect your databases by putting them in a virtual private cloud.

DB instances

The basic building block of Amazon RDS is the DB instance. A *DB instance* is an isolated database environment in the AWS Cloud. Your DB instance can contain multiple user-created databases. You can access your DB instance by using the same tools and applications that you use with a standalone database instance. You can create and modify a DB instance by using the AWS Command Line Interface, the Amazon RDS API, or the AWS Management Console.

Each DB instance runs a *DB engine*. Amazon RDS currently supports the MySQL, MariaDB, PostgreSQL, Oracle, and Microsoft SQL Server DB engines. Each DB engine has its own supported features, and each version of a DB engine may include specific features. Additionally, each DB engine has a set of parameters in a DB parameter group that control the behavior of the databases that it manages.

The computation and memory capacity of a DB instance is determined by its *DB instance class*. You can select the DB instance that best meets your needs. If your needs change over time, you can change DB instances.

DB instances

DB instance storage comes in three types: Magnetic, General Purpose (SSD), and Provisioned IOPS (PIOPS). They differ in performance characteristics and price, allowing you to tailor your storage performance and cost to the needs of your database. Each DB instance has minimum and maximum storage requirements depending on the storage type and the database engine it supports. It's important to have sufficient storage so that your databases have room to grow. Also, sufficient storage makes sure that features for the DB engine have room to write content or log entries.

You can run a DB instance on a virtual private cloud (VPC) using the Amazon Virtual Private Cloud (Amazon VPC) service. When you use a VPC, you have control over your virtual networking environment. You can choose your own IP address range, create subnets, and configure routing and access control lists. The basic functionality of Amazon RDS is the same whether it's running in a VPC or not. Amazon RDS manages backups, software patching, automatic failure detection, and recovery. There's no additional cost to run your DB instance in a VPC.

AWS Regions and Availability Zones

Amazon cloud computing resources are housed in highly available data center facilities in different areas of the world (for example, North America, Europe, or Asia). Each data center location is called an AWS Region.

Each AWS Region contains multiple distinct locations called Availability Zones, or AZs. Each Availability Zone is engineered to be isolated from failures in other Availability Zones. Each is engineered to provide inexpensive, low-latency network connectivity to other Availability Zones in the same AWS Region. By launching instances in separate Availability Zones, you can protect your applications from the failure of a single location.

You can run your DB instance in several Availability Zones, an option called a Multi-AZ deployment. When you choose this option, Amazon automatically provisions and maintains a secondary standby DB instance in a different Availability Zone. Your primary DB instance is synchronously replicated across Availability Zones to the secondary instance. This approach helps provide data redundancy and failover support, eliminate I/O freezes, and minimize latency spikes during system backups.

Security

A security group controls the access to a DB instance. It does so by allowing access to IP address ranges or Amazon EC2 instances that you specify.

Monitoring an Amazon RDS DB instance

There are several ways that you can track the performance and health of a DB instance. You can use the Amazon CloudWatch service to monitor the performance and health of a DB instance. CloudWatch performance charts are shown in the Amazon RDS console. You can also subscribe to Amazon RDS events to be notified about changes to a DB instance, DB snapshot, DB parameter group, or DB security group.

How to work with Amazon RDS

There are several ways that you can interact with Amazon RDS.

AWS Management Console

The AWS Management Console is a simple web-based user interface. You can manage your DB instances from the console with no programming required. To access the Amazon RDS console, sign in to the AWS Management Console and open the Amazon RDS console at <https://console.aws.amazon.com/rds/>.

Command line interface

You can use the AWS Command Line Interface (AWS CLI) to access the Amazon RDS API interactively.

How to work with Amazon RDS

Programming with Amazon RDS

If you are a developer, you can access the Amazon RDS programmatically.

For application development, we recommend that you use one of the AWS Software Development Kits (SDKs). The AWS SDKs handle low-level details such as authentication, retry logic, and error handling, so that you can focus on your application logic. AWS SDKs are available for a wide variety of languages.

AWS also provides libraries, sample code, tutorials, and other resources to help you get started more easily.



Amazon RDS DB instances

Amazon RDS DB instances

A DB instance is an isolated database environment running in the cloud. It is the basic building block of Amazon RDS. A DB instance can contain multiple user-created databases, and can be accessed using the same client tools and applications you might use to access a standalone database instance. DB instances are simple to create and modify with the Amazon AWS command line tools, Amazon RDS API operations, or the AWS Management Console.

You can have up to 40 Amazon RDS DB instances, with the following limitations:

- 10 for each SQL Server edition (Enterprise, Standard, Web, and Express) under the "license-included" model
- 10 for Oracle under the "license-included" model
- 40 for MySQL, MariaDB, or PostgreSQL
- 40 for Oracle under the "bring-your-own-license" (BYOL) licensing model

Amazon RDS DB instances

Each DB instance has a DB instance identifier. This customer-supplied name uniquely identifies the DB instance when interacting with the Amazon RDS API and AWS CLI commands. The DB instance identifier must be unique for that customer in an AWS Region.

The identifier is used as part of the DNS hostname allocated to your instance by RDS. For example, if you specify `db1` as the DB instance identifier, then RDS will automatically allocate a DNS endpoint for your instance, such as `db1.123456789012.us-east-1.rds.amazonaws.com`, where `123456789012` is the fixed identifier for a specific region for your account.

Each DB instance supports a database engine. Amazon RDS currently supports MySQL, MariaDB, PostgreSQL, Oracle, Microsoft SQL Server, and Amazon Aurora database engines.

Amazon RDS DB instances

When creating a DB instance, some database engines require that a database name be specified. A DB instance can host multiple databases, or a single Oracle database with multiple schemas. The database name value depends on the database engine:

- For the MySQL and MariaDB database engines, the database name is the name of a database hosted in your DB instance. Databases hosted by the same DB instance must have a unique name within that instance.
- For the Oracle database engine, database name is used to set the value of `ORACLE_SID`, which must be supplied when connecting to the Oracle RDS instance.
- For the Microsoft SQL Server database engine, database name is not a supported parameter.
- For the PostgreSQL database engine, the database name is the name of a database hosted in your DB instance. A database name is not required when creating a DB instance. Databases hosted by the same DB instance must have a unique name within that instance.

Amazon RDS DB instances

Amazon RDS creates a master user account for your DB instance as part of the creation process. This master user has permissions to create databases and to perform create, delete, select, update, and insert operations on tables the master user creates. You must set the master user password when you create a DB instance, but you can change it at any time using the Amazon AWS command line tools, Amazon RDS API operations, or the AWS Management Console. You can also change the master user password and manage users using standard SQL commands.



DB instance classes

DB instance class types

Amazon RDS supports three types of instance classes: Standard, Memory Optimized, and Burstable Performance. The following are the Standard DB instance classes available:

- **db.m5** – Latest-generation general-purpose instance classes that provide a balance of compute, memory, and network resources, and are a good choice for many applications. The db.m5 instance classes provide more computing capacity than the previous db.m4 instance classes. They are powered by the AWS Nitro System, a combination of dedicated hardware and lightweight hypervisor.
- **db.m4** – Current-generation general-purpose instance classes that provide more computing capacity than the previous db.m3 instance classes.
- **db.r5** – Latest-generation instance classes optimized for memory-intensive applications. These offer improved networking and Amazon Elastic Block Store (Amazon EBS) performance. They are powered by the AWS Nitro System, a combination of dedicated hardware and lightweight hypervisor.
- **db.r4** – Current-generation instance classes optimized for memory-intensive applications. These offer improved networking and Amazon EBS performance.
- **db.t3** – Latest-generation instance classes that provide a baseline performance level, with the ability to burst to full CPU usage. These instance classes provide more computing capacity than the previous db.t2 instance classes. They are powered by the AWS Nitro System, a combination of dedicated hardware and lightweight hypervisor.

Supported DB engines for DB instance classes

The following are DB engine considerations for DB instance classes:

MariaDB

The Graviton2 instance classes db.m6g and db.r6g are supported for all MariaDB 10.5 versions and MariaDB version 10.4.13 and higher 10.4 versions.

Microsoft SQL Server

Instance class support varies according to the version and edition of SQL Server.

MySQL

The Graviton2 instance classes db.m6g and db.r6g are supported for Amazon RDS MySQL versions 8.0.17 and higher.

Oracle

Instance class support varies according to the version and edition of Oracle.

PostgreSQL

The Graviton2 instance classes db.m6g and db.r6g are supported for Amazon RDS PostgreSQL versions 12.3 and higher.

Supported DB engines for DB instance classes

In the following table, you can find details about supported Amazon RDS DB instance classes for each Amazon RDS DB engine.

Instance class	MariaDB	Microsoft SQL Server	MySQL	Oracle	PostgreSQL
db.m5 – Latest generation standard instance classes					
db.m5.24xlarge	Yes	Yes	Yes	Yes	Yes
db.m5.16xlarge	Yes	Yes	Yes	Yes	PostgreSQL 11.6 & higher, 10.11 & higher, 9.6.16 & higher
db.m5.12xlarge	Yes	Yes	Yes	Yes	Yes
db.m5.8xlarge	Yes	Yes	Yes	Yes	PostgreSQL 11.6 & higher, 10.11 & higher, 9.6.16 & higher
db.m5.4xlarge	Yes	Yes	Yes	Yes	Yes
db.m5.2xlarge	Yes	Yes	Yes	Yes	Yes
db.m5.xlarge	Yes	Yes	Yes	Yes	Yes
db.m5.large	Yes	Yes	Yes	Yes	Yes
db.m4 – Current generation standard instance classes					
db.m4.16xlarge	Yes	Yes	MySQL 8.0, 5.7, 5.6	Yes	Yes
db.m4.10xlarge	Yes	Yes	Yes	Yes	Yes
db.m4.4xlarge	Yes	Yes	Yes	Yes	Yes
db.m4.2xlarge	Yes	Yes	Yes	Yes	Yes
db.m4.xlarge	Yes	Yes	Yes	Yes	Yes
db.m4.large	Yes	Yes	Yes	Yes	Yes

Supported DB engines for DB instance classes

In the following table, you can find details about supported Amazon RDS DB instance classes for each Amazon RDS DB engine.

Instance class	MariaDB	Microsoft SQL Server	MySQL	Oracle	PostgreSQL
db.r5 – Latest generation memory optimized instance classes					
db.r5.24xlarge	Yes	Yes	Yes	Yes	Yes
db.r5.16xlarge	Yes	Yes	Yes	Yes	PostgreSQL 11.6 & higher, 10.11 & higher, 9.6.16 & higher
db.r5.12xlarge	Yes	Yes	Yes	Yes	Yes
db.r5.8xlarge	Yes	Yes	Yes	Yes	PostgreSQL 11.6 & higher, 10.11 & higher, 9.6.16 & higher
db.r5.4xlarge	Yes	Yes	Yes	Yes	Yes
db.r5.2xlarge	Yes	Yes	Yes	Yes	Yes
db.r5.xlarge	Yes	Yes	Yes	Yes	Yes
db.r5.large	Yes	Yes	Yes	Yes	Yes
db.r4 – Current generation memory optimized instance classes					
db.r4.16xlarge	Yes	Yes	MySQL 8.0, 5.7, 5.6	Yes	Yes
db.r4.8xlarge	Yes	Yes	MySQL 8.0, 5.7, 5.6	Yes	Yes
db.r4.4xlarge	Yes	Yes	MySQL 8.0, 5.7, 5.6	Yes	Yes
db.r4.2xlarge	Yes	Yes	MySQL 8.0, 5.7, 5.6	Yes	Yes
db.r4.xlarge	Yes	Yes	MySQL 8.0, 5.7, 5.6	Yes	Yes
db.r4.large	Yes	Yes	MySQL 8.0, 5.7, 5.6	Yes	Yes

Supported DB engines for DB instance classes

In the following table, you can find details about supported Amazon RDS DB instance classes for each Amazon RDS DB engine.

Instance class	MariaDB	Microsoft SQL Server	MySQL	Oracle	PostgreSQL
db.t3 – Latest generation burstable performance instance classes					
db.t3.xlarge	Yes	Yes	Yes	Yes	Yes
db.t3.xlarge	Yes	Yes	Yes	Yes	Yes
db.t3.large	Yes	Yes	Yes	Yes	Yes
db.t3.medium	Yes	Yes	Yes	Yes	Yes
db.t3.small	Yes	Yes	Yes	Yes	Yes
db.t3.micro	Yes	No	Yes	Yes	Yes

Changing your DB instance class

You can change the CPU and memory available to a DB instance by changing its DB instance class. To change the DB instance class, modify your DB instance by following the instructions in [Modifying an Amazon RDS DB instance](#).

Some instance classes require that your DB instance is in a VPC. If your current DB instance isn't in a VPC, and you want to use an instance class that requires one, first move your DB instance into a VPC.

Configuring the processor for a DB instance class

Amazon RDS DB instance classes support Intel Hyper-Threading Technology, which enables multiple threads to run concurrently on a single Intel Xeon CPU core. Each thread is represented as a virtual CPU (vCPU) on the DB instance. A DB instance has a default number of CPU cores, which varies according to DB instance type. For example, a db.m4.xlarge DB instance type has two CPU cores and two threads per core by default—four vCPUs in total.

Overview of configuring the processor

In most cases, you can find a DB instance class that has a combination of memory and number of vCPUs to suit your workloads. However, you can also specify the following processor features to optimize your DB instance for specific workloads or business needs:

- **Number of CPU cores** – You can customize the number of CPU cores for the DB instance. You might do this to potentially optimize the licensing costs of your software with a DB instance that has sufficient amounts of RAM for memory-intensive workloads but fewer CPU cores.
- **Threads per core** – You can disable Intel Hyper-Threading Technology by specifying a single thread per CPU core. You might do this for certain workloads, such as high-performance computing (HPC) workloads.

You can control the number of CPU cores and threads for each core separately. You can set one or both in a request. After a setting is associated with a DB instance, the setting persists until you change it.

The processor settings for a DB instance are associated with snapshots of the DB instance. When a snapshot is restored, its restored DB instance uses the processor feature settings used when the snapshot was taken.

Overview of configuring the processor

If you modify the DB instance class for a DB instance with nondefault processor settings, either specify default processor settings or explicitly specify processor settings at modification. This requirement ensures that you are aware of the third-party licensing costs that might be incurred when you modify the DB instance.

There is no additional or reduced charge for specifying processor features on an Amazon RDS DB instance. You're charged the same as for DB instances that are launched with default CPU configurations.

CPU cores and threads per CPU core per DB instance class

In the following table, you can find the DB instance classes that support setting a number of CPU cores and CPU threads per core. You can also find the default value and the valid values for the number of CPU cores and CPU threads per core for each DB instance class.

DB instance class	Default vCPUs	Default CPU cores	Default threads per core	Valid number of CPU cores	Valid number of threads per core
db.m5.large	2	1	2	1	1, 2
db.m5.xlarge	4	2	2	2	1, 2
db.m5.2xlarge	8	4	2	2, 4	1, 2
db.m5.4xlarge	16	8	2	2, 4, 6, 8	1, 2
db.m5.8xlarge	32	16	2	2, 4, 6, 8, 10, 12, 14, 16	1, 2
db.m5.12xlarge	48	24	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24	1, 2
db.m5.16xlarge	64	32	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32	1, 2
db.m5.24xlarge	96	48	2	4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48	1, 2
db.m4.10xlarge	40	20	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20	1, 2
db.m4.16xlarge	64	32	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32	1, 2

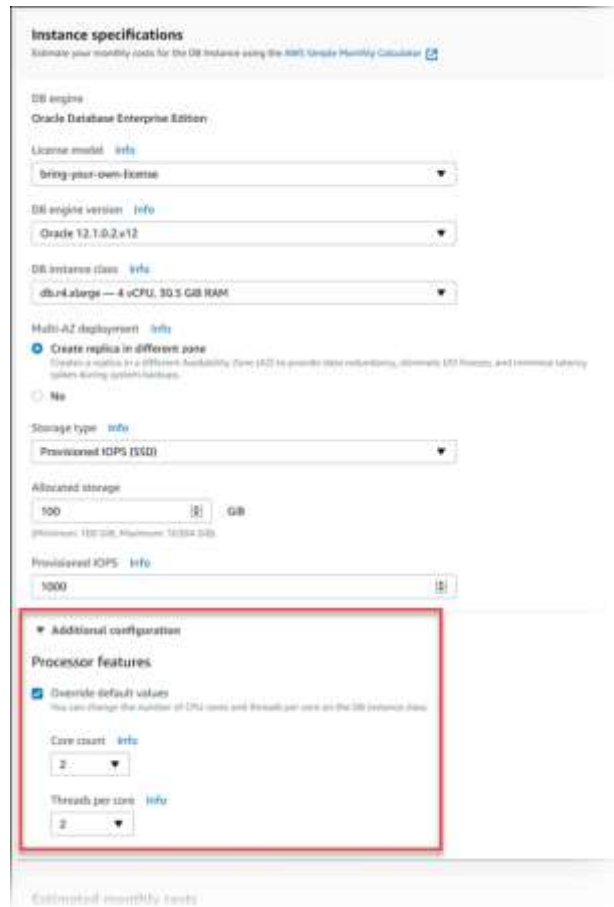
CPU cores and threads per CPU core per DB instance class

DB instance class	Default vCPUs	Default CPU cores	Default threads per core	Valid number of CPU cores	Valid number of threads per core
db.r5.large	2	1	2	1	1, 2
db.r5.xlarge	4	2	2	2	1, 2
db.r5.2xlarge	8	4	2	2, 4	1, 2
db.r5.4xlarge	16	8	2	2, 4, 6, 8	1, 2
db.r5.8xlarge	32	16	2	2, 4, 6, 8, 10, 12, 14, 16	1, 2
db.r5.12xlarge	48	24	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24	1, 2
db.r5.16xlarge	64	32	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32	1, 2
db.r5.24xlarge	96	48	2	4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48	1, 2
db.r5b.large	2	1	2	1	1, 2
db.r5b.xlarge	4	2	2	2	1, 2
db.r5b.2xlarge	8	4	2	2, 4	1, 2
db.r5b.4xlarge	16	8	2	2, 4, 6, 8	1, 2
db.r5b.8xlarge	32	16	2	2, 4, 6, 8, 10, 12, 14, 16	1, 2
db.r5b.12xlarge	48	24	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24	1, 2
db.r5b.16xlarge	64	32	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32	1, 2
db.r5b.24xlarge	96	48	2	4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48	1, 2
db.r4.large	2	1	2	1	1, 2
db.r4.xlarge	4	2	2	1, 2	1, 2
db.r4.2xlarge	8	4	2	1, 2, 3, 4	1, 2
db.r4.4xlarge	16	8	2	1, 2, 3, 4, 5, 6, 7, 8	1, 2
db.r4.8xlarge	32	16	2	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16	1, 2
db.r4.16xlarge	64	32	2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32	1, 2

Setting the CPU cores and threads per CPU core for a DB instance class

You can set the CPU cores and the threads per CPU core for a DB instance class using the AWS Management Console, the AWS CLI, or the RDS API.

Console



Instance specifications
Estimate your monthly costs for the DB instance using the [AWS Simple Monthly Calculator](#).

DB engine
Oracle Database Enterprise Edition

License model: [info](#)
bring-your-own-license

DB engine version: [info](#)
Oracle 12.1.0.2.x12

DB instance class: [info](#)
db.r4.large — 4 vCPU, 30.5 GB RAM

Multi-AZ deployment [info](#)
☒ Create replica in different zone
Create a replica in a different Availability Zone (AZ) to provide high availability, eliminate RTO, and prevent latency spikes during system updates.
☐ No

Storage type [info](#)
Provisioned IOPS (SSD)

Allocated storage
500 GB
(Maximum: 1 TB; Minimum: 10 GB)

Provisioned IOPS [info](#)
3000

Additional configuration

Processor features

☒ Override default values
You can change the number of CPU cores and threads per core on the DB instance class.

Core count: [info](#)
2

Threads per core: [info](#)
2

Estimated monthly costs

Setting the CPU cores and threads per CPU core for a DB instance class

Set the following options to the appropriate values for your DB instance class under **Processor features**:

- **Core count** – Set the number of CPU cores using this option. The value must be equal to or less than the maximum number of CPU cores for the DB instance class.
- **Threads per core** – Specify **2** to enable multiple threads per core, or specify **1** to disable multiple threads per core.

When you modify or restore a DB instance, you can also set the CPU cores and the threads per CPU core to the defaults for the instance class.

When you view the details for a DB instance in the console, you can view the processor information for its DB instance class on the **Configuration** tab. The following image shows a DB instance class with one CPU core and multiple threads per core enabled.

Instance and IOPS	
Instance Class	db.r4.large
Core count	1
Threads per core	2
vCPU enabled	2
Storage Type	Provisioned IOPS (SSD)
IOPS	1000
Storage	100 GiB

Setting the CPU cores and threads per CPU core for a DB instance class

AWS CLI

You can set the processor features for a DB instance when you run one of the following AWS CLI commands:

- `create-db-instance`
- `modify-db-instance`
- `restore-db-instance-from-db-snapshot`
- `restore-db-instance-from-s3`
- `restore-db-instance-to-point-in-time`

To configure the processor of a DB instance class for a DB instance by using the AWS CLI, include the `--processor-features` option in the command. Specify the number of CPU cores with the `coreCount` feature name, and specify whether multiple threads per core are enabled with the `threadsPerCore` feature name.

Setting the CPU cores and threads per CPU core for a DB instance class

The option has the following syntax.

```
--processor-features "Name=coreCount,Value=<value>" "Name=threadsPerCore,Value=<value>"
```

Setting the number of CPU cores for a DB instance

The following example modifies mydbinstance by setting the number of CPU cores to 4. The changes are applied immediately by using `--apply-immediately`. If you want to apply the changes during the next scheduled maintenance window, omit the `--apply-immediately` option.

For Linux, macOS, or Unix:

```
aws rds modify-db-instance \  
  --processor-features "Name=coreCount,Value=4" \  
  --apply-immediately
```

Setting the CPU cores and threads per CPU core for a DB instance class

Setting the number of CPU cores for a DB instance

For Windows:

```
aws rds modify-db-instance ^  
  --processor-features "Name=coreCount,Value=4" ^  
  --apply-immediately
```

Setting the number of CPU cores and disabling multiple threads for a DB instance

The following example modifies `mydbinstance` by setting the number of CPU cores to `4` and disabling multiple threads per core. The changes are applied immediately by using `--apply-immediately`. If you want to apply the changes during the next scheduled maintenance window, omit the `--apply-immediately` option.

For Linux, macOS, or Unix:

```
aws rds modify-db-instance \  
  --processor-features "Name=coreCount,Value=4" "Name=threadsPerCore,Value=1" \  
  --apply-immediately
```

For Windows:

```
aws rds modify-db-instance ^  
  --processor-features "Name=coreCount,Value=4" "Name=threadsPerCore,Value=1" ^  
  --apply-immediately
```

Viewing the valid processor values for a DB instance class

You can view the valid processor values for a particular DB instance class by running the `describe-orderable-db-instance-options` command and specifying the instance class for the `--db-instance-class` option. For example, the output for the following command shows the processor options for the `db.r3.large` instance class.

```
aws rds describe-orderable-db-instance-options --engine oracle-ee --db-instance-class db.r3.large
```

Viewing the valid processor values for a DB instance class

Following is sample output for the command in JSON format.

```
{
  "SupportsIops": true,
  "MaxIopsPerGih": 58.0,
  "LicenseModel": "bring-your-own-license",
  "DbInstanceClass": "db.r3.large",
  "SupportsIAMDatabaseAuthentication": false,
  "MinStorageSize": 100,
  "AvailabilityZones": [
    {
      "Name": "us-west-2a"
    },
    {
      "Name": "us-west-2b"
    },
    {
      "Name": "us-west-2c"
    }
  ],
  "EngineVersion": "12.1.0.2.v2",
  "MaxStorageSize": 32768,
  "MinIopsPerGih": 1.0,
  "MaxIopsPerDbInstance": 40000,
  "ReadReplicaCapable": false,
  "AvailableProcessorFeatures": [
    {
      "Name": "coreCount",
      "DefaultValue": "1",
      "AllowedValues": "1"
    },
    {
      "Name": "threadsPerCore",
      "DefaultValue": "2",
      "AllowedValues": "1,2"
    }
  ],
  "SupportsEnhancedMonitoring": true,
  "SupportsPerformanceInsights": false,
  "MinIopsPerDbInstance": 1000,
  "StorageType": "io1",
  "Vpc": false,
  "SupportsStorageEncryption": true,
  "Engine": "oracle-se",
  "MultiAZCapable": true
}
```

Viewing the valid processor values for a DB instance class

In addition, you can run the following commands for DB instance class processor information:

- `describe-db-instances` – Shows the processor information for the specified DB instance.
- `describe-db-snapshots` – Shows the processor information for the specified DB snapshot.
- `describe-valid-db-instance-modifications` – Shows the valid modifications to the processor for the specified DB instance.

In the output of the preceding commands, the values for the processor features are not null only if the following conditions are met:

- You are using an Oracle DB instance.
- Your Oracle DB instance supports changing processor values.
- The current CPU core and thread settings are set to nondefault values.

If the preceding conditions aren't met, you can get the instance type using [describe-db-instances](#). You can get the processor information for this instance type by running the EC2 operation [describe-instance-types](#).

Returning to default processor settings for a DB instance

The following example modifies `mydbinstance` by returning its DB instance class to the default processor values for it. The changes are applied immediately by using `--apply-immediately`. If you want to apply the changes during the next scheduled maintenance window, omit the `--apply-immediately` option.

For Linux, macOS, or Unix:

```
aws rds modify-db-instance \  
  --use-default-processor-features \  
  --apply-immediately
```

For Windows:

```
aws rds modify-db-instance ^  
  --use-default-processor-features ^  
  --apply-immediately
```

Returning to the default number of CPU cores for a DB instance

The following example modifies `mydbinstance` by returning its DB instance class to the default number of CPU cores for it. The threads per core setting isn't changed. The changes are applied immediately by using `--apply-immediately`. If you want to apply the changes during the next scheduled maintenance window, omit the `--apply-immediately` option.

Example 1: Apply immediately

```
aws rds modify-db-instance \  
  --processor-features "Name=coreCount,Value=DEFAULT" \  
  --apply-immediately
```

Example 2: Apply later

```
aws rds modify-db-instance ^  
  --processor-features "Name=coreCount,Value=DEFAULT" ^  
  --apply-immediately
```

Returning to the default number of threads per core for a DB instance

The following example modifies `mydbinstance` by returning its DB instance class to the default number of threads per core for it. The number of CPU cores setting isn't changed. The changes are applied immediately by using `--apply-immediately`. If you want to apply the changes during the next scheduled maintenance window, omit the `--apply-immediately` option.

For Linux, macOS, or Unix:

```
aws rds modify-db-instance \  
  --processor-features "Name=threadsPerCore,Value=DEFAULT" \  
  --apply-immediately
```

For Windows:

```
aws rds modify-db-instance ^  
  --processor-features "Name=threadsPerCore,Value=DEFAULT" ^  
  --apply-immediately
```

RDS API

To configure the processor features of a DB instance class for a DB instance by using the Amazon RDS API, include the `ProcessFeatures` parameter in the call.

The parameter has the following syntax.

```
ProcessFeatures "Name=coreCount,Value=<value>" "Name=threadsPerCore,Value=<value>"
```

Specify the number of CPU cores with the `coreCount` feature name, and specify whether multiple threads per core are enabled with the `threadsPerCore` feature name.

You can view the valid processor values for a particular instance class by running the [DescribeOrderableDBInstanceOptions](#) operation and specifying the instance class for the `DBInstanceClass` parameter.

You can also use the following operations:

[DescribeDBInstances](#) – Shows the processor information for the specified DB instance.

[DescribeDBSnapshots](#) – Shows the processor information for the specified DB snapshot.

[DescribeValidDBInstanceModifications](#) – Shows the valid modifications to the processor for the specified DB instance.

RDS API

In the output of the preceding operations, the values for the processor features are not null only if the following conditions are met:

- You are using an Oracle DB instance.
- Your Oracle DB instance supports changing processor values.
- The current CPU core and thread settings are set to nondefault values.

Hardware specifications for DB instance classes

The following terminology is used to describe hardware specifications for DB instance classes:

vCPU

The number of virtual central processing units (CPUs). A *virtual CPU* is a unit of capacity that you can use to compare DB instance classes. Instead of purchasing or leasing a particular processor to use for several months or years, you are renting capacity by the hour. Our goal is to make a consistent and specific amount of CPU capacity available, within the limits of the actual underlying hardware.

ECU

The relative measure of the integer processing power of an Amazon EC2 instance. To make it easy for developers to compare CPU capacity between different instance classes, we have defined an Amazon EC2 Compute Unit. The amount of CPU that is allocated to a particular instance is expressed in terms of these EC2 Compute Units. One ECU currently provides CPU capacity equivalent to a 1.0–1.2 GHz 2007 Opteron or 2007 Xeon processor.

Hardware specifications for DB instance classes

Memory (GiB)

The RAM, in gibibytes, allocated to the DB instance. There is often a consistent ratio between memory and vCPU. As an example, take the db.r4 instance class, which has a memory to vCPU ratio similar to the db.r5 instance class. However, for most use cases the db.r5 instance class provides better, more consistent performance than the db.r4 instance class.

VPC Only

The instance class is supported only for DB instances that are in a VPC based on the Amazon VPC service. In some cases, you might want to use an instance class that requires a VPC but your current DB instance isn't in a VPC. In these cases, start by moving your DB instance into a VPC.

Hardware specifications for DB instance classes

EBS-Optimized

The DB instance uses an optimized configuration stack and provides additional, dedicated capacity for I/O. This optimization provides the best performance by minimizing contention between I/O and other traffic from your instance. For more information about Amazon EBS-optimized instances, see [Amazon EBS-Optimized instances](#) in the *Amazon EC2 User Guide for Linux Instances*.

Max. Bandwidth (Mbps)

The maximum bandwidth in megabits per second. Divide by 8 to get the expected throughput in megabytes per second.

Network Performance

The network speed relative to other DB instance classes.

Hardware specifications for DB instance classes

In the following table, you can find hardware details about the Amazon RDS DB instance classes .

Instance class	vCPU	ECU	Memory (GiB)	VPC only	EBS optimized	Max. bandwidth (mbps)	Network performance
db.m5 – Latest generation standard instance classes							
db.m5.24xlarge	96	345	584	Yes	Yes	19,000	25 Gbps
db.m5.16xlarge	64	262	256	Yes	Yes	13,600	20 Gbps
db.m5.12xlarge	48	173	192	Yes	Yes	9,500	10 Gbps
db.m5.8xlarge	32	131	128	Yes	Yes	6,800	10 Gbps
db.m5.4xlarge	16	61	64	Yes	Yes	4,750	Up to 10 Gbps
db.m5.2xlarge*	8	31	32	Yes	Yes	Up to 4,750	Up to 10 Gbps
db.m5.xlarge*	4	15	16	Yes	Yes	Up to 4,750	Up to 10 Gbps
db.m5.large*	2	10	8	Yes	Yes	Up to 4,750	Up to 10 Gbps
db.m4 – Current generation standard instance classes							
db.m4.16xlarge	64	188	256	Yes	Yes	10,000	25 Gbps
db.m4.10xlarge	40	124.5	160	Yes	Yes	4,000	10 Gbps
db.m4.4xlarge	16	53.5	64	Yes	Yes	2,000	High
db.m4.2xlarge	8	25.5	32	Yes	Yes	1,000	High
db.m4.xlarge	4	13	16	Yes	Yes	750	High
db.m4.large	2	6.5	8	Yes	Yes	450	Moderate

Hardware specifications for DB instance classes

Instance class	vCPU	ECU	Memory (GiB)	VPC only	EBS optimized	Max. bandwidth (mbps)	Network performance
db.r5 – Latest generation memory optimized instance classes							
db.r5.24xlarge	96	347	768	Yes	Yes	19,000	25 Gbps
db.r5.16xlarge	64	264	512	Yes	Yes	13,600	20 Gbps
db.r5.12xlarge	48	173	384	Yes	Yes	9,500	10 Gbps
db.r5.8xlarge	32	132	256	Yes	Yes	6,800	10 Gbps
db.r5.4xlarge	16	71	128	Yes	Yes	4,750	Up to 10 Gbps
db.r5.2xlarge*	8	38	64	Yes	Yes	Up to 4,750	Up to 10 Gbps
db.r5.xlarge*	4	19	32	Yes	Yes	Up to 4,750	Up to 10 Gbps
db.r5.large*	2	10	16	Yes	Yes	Up to 4,750	Up to 10 Gbps
db.r4 – Current generation memory optimized instance classes							
db.r4.16xlarge	64	195	488	Yes	Yes	14,000	25 Gbps
db.r4.8xlarge	32	99	244	Yes	Yes	7,000	10 Gbps
db.r4.4xlarge	16	53	122	Yes	Yes	3,500	Up to 10 Gbps
db.r4.2xlarge	8	27	61	Yes	Yes	1,700	Up to 10 Gbps
db.r4.xlarge	4	13.5	30.5	Yes	Yes	850	Up to 10 Gbps
db.r4.large	2	7	15.25	Yes	Yes	425	Up to 10 Gbps

Hardware specifications for DB instance classes

Instance class	vCPU	ECU	Memory (GiB)	VPC only	EBS optimized	Max. bandwidth (mbps)	Network performance
db.t3 – Latest generation burstable performance instance classes							
db.t3.2xlarge*	8	Variable	32	Yes	Yes	Up to 2,048	Up to 5 Gbps
db.t3.xlarge*	4	Variable	16	Yes	Yes	Up to 2,048	Up to 5 Gbps
db.t3.large*	2	Variable	8	Yes	Yes	Up to 2,048	Up to 5 Gbps
db.t3.medium*	2	Variable	4	Yes	Yes	Up to 1,536	Up to 5 Gbps
db.t3.small*	2	Variable	2	Yes	Yes	Up to 1,536	Up to 5 Gbps
db.t3.micro*	2	Variable	1	Yes	Yes	Up to 1,536	Up to 5 Gbps



Amazon RDS DB instance storage



Amazon RDS storage types

DB instances for Amazon RDS for MySQL, MariaDB, PostgreSQL, Oracle, and Microsoft SQL Server use Amazon Elastic Block Store (Amazon EBS) volumes for database and log storage. Depending on the amount of storage requested, Amazon RDS automatically stripes across multiple Amazon EBS volumes to enhance performance.

Amazon RDS provides three storage types: General Purpose SSD (also known as gp2), Provisioned IOPS SSD (also known as io1), and magnetic (also known as standard). They differ in performance characteristics and price, which means that you can tailor your storage performance and cost to the needs of your database workload. You can create MySQL, MariaDB, Oracle, and PostgreSQL RDS DB instances with up to 64 terabytes (TiB) of storage. You can create SQL Server RDS DB instances with up to 16 TiB of storage. For this amount of storage, use the Provisioned IOPS SSD and General Purpose SSD storage types.

Amazon RDS storage types

The following list briefly describes the three storage types:

- **General Purpose SSD** – General Purpose SSD volumes offer cost-effective storage that is ideal for a broad range of workloads. These volumes deliver single-digit millisecond latencies and the ability to burst to 3,000 IOPS for extended periods of time. Baseline performance for these volumes is determined by the volume's size.
- **Provisioned IOPS** – Provisioned IOPS storage is designed to meet the needs of I/O-intensive workloads, particularly database workloads, that require low I/O latency and consistent I/O throughput.
- **Magnetic** – Amazon RDS also supports magnetic storage for backward compatibility. We recommend that you use General Purpose SSD or Provisioned IOPS for any new storage needs. The maximum amount of storage allowed for DB instances on magnetic storage is less than that of the other storage types.

Several factors can affect the performance of Amazon EBS volumes, such as instance configuration, I/O characteristics, and workload demand.

General Purpose SSD storage

General Purpose SSD storage offers cost-effective storage that is acceptable for most database workloads. The following are the storage size ranges for General Purpose SSD DB instances:

- MariaDB, MySQL, Oracle, and PostgreSQL database instances: 20 GiB–64 TiB
- SQL Server for Enterprise, Standard, Web, and Express editions: 20 GiB–16 TiB

Baseline I/O performance for General Purpose SSD storage is 3 IOPS for each GiB, with a minimum of 100 IOPS. This relationship means that larger volumes have better performance. For example, baseline performance for a 100-GiB volume is 300 IOPS. Baseline performance for a 1-TiB volume is 3,000 IOPS. And baseline performance for a 5.34-TiB volume is 16,000 IOPS.

Volumes below 1 TiB in size also have ability to burst to 3,000 IOPS for extended periods of time. Burst is not relevant for volumes above 1 TiB. Instance I/O credit balance determines burst performance.

Many workloads never deplete the burst balance, making General Purpose SSD an ideal storage choice for many workloads. However, some workloads can exhaust the 3,000 IOPS burst storage credit balance, so you should plan your storage capacity to meet the needs of your workloads.

I/O credits and burst performance

General Purpose SSD storage performance is governed by volume size, which dictates the base performance level of the volume and how quickly it accumulates I/O credits. Larger volumes have higher base performance levels and accumulate I/O credits faster. *I/O credits* represent the available bandwidth that your General Purpose SSD storage can use to burst large amounts of I/O when more than the base level of performance is needed. The more I/O credits your storage has for I/O, the more time it can burst beyond its base performance level and the better it performs when your workload requires more performance.

When using General Purpose SSD storage, your DB instance receives an initial I/O credit balance of 5.4 million I/O credits. This initial credit balance is enough to sustain a burst performance of 3,000 IOPS for 30 minutes. This balance is designed to provide a fast initial boot cycle for boot volumes and to provide a good bootstrapping experience for other applications. Volumes earn I/O credits at the baseline performance rate of 3 IOPS for each GiB of volume size. For example, a 100-GiB SSD volume has a baseline performance of 300 IOPS.

I/O credits and burst performance

When your storage requires more than the base performance I/O level, it uses I/O credits in the I/O credit balance to burst to the required performance level. Such a burst goes to a maximum of 3,000 IOPS. Storage larger than 1,000 GiB has a base performance that is equal or greater than the maximum burst performance. When your storage uses fewer I/O credits than it earns in a second, unused I/O credits are added to the I/O credit balance. The maximum I/O credit balance for a DB instance using General Purpose SSD storage is equal to the initial I/O credit balance (5.4 million I/O credits).

Suppose that your storage uses all of its I/O credit balance. If so, its maximum performance remains at the base performance level until I/O demand drops below the base level and unused I/O credits are added to the I/O credit balance. (The *base performance level* is the rate at which your storage earns I/O credits.) The more storage, the greater the base performance is and the faster it replenishes the I/O credit balance.

I/O credits and burst performance

The following table lists several storage sizes. For each storage size, it lists the associated base performance of the storage, which is also the rate at which it accumulates I/O credits. The table also lists the burst duration at the 3,000 IOPS maximum, when starting with a full I/O credit balance. In addition, the table lists the time in seconds that the storage takes to refill an empty I/O credit balance.

Storage size (GiB)	Base performance (IOPS)	Maximum burst duration at 3,000 IOPS (seconds)	Seconds to fill empty I/O credit balance
1	100	1,862	54,000
100	300	2,000	18,000
250	750	2,400	7,200
500	1,500	3,600	3,600
750	2,250	7,200	2,400
1,000	3,000	Infinite	N/A
5,334	16,000	N/A	N/A

I/O credits and burst performance

The burst duration of your storage depends on the size of the storage, the burst IOPS required, and the I/O credit balance when the burst begins. This relationship is shown in the equation following.

$$\text{Burst duration} = \frac{(\text{Credit balance})}{(\text{Burst IOPS}) - 3 * (\text{Storage size in GiB})}$$

You might notice that your storage performance is frequently limited to the base level due to an empty I/O credit balance. If so, consider allocating more General Purpose SSD storage with a higher base performance level.

Alternatively, you can switch to Provisioned IOPS storage for workloads that require sustained IOPS performance.

For workloads with steady state I/O requirements, provisioning less than 100 GiB of General Purpose SSD storage might result in higher latencies if you exhaust your I/O credit balance.

Provisioned IOPS SSD storage

For a production application that requires fast and consistent I/O performance, we recommend Provisioned IOPS (input/output operations per second) storage. Provisioned IOPS storage is a storage type that delivers predictable performance, and consistently low latency. Provisioned IOPS storage is optimized for online transaction processing (OLTP) workloads that have consistent performance requirements. Provisioned IOPS helps performance tuning of these workloads.

When you create a DB instance, you specify the IOPS rate and the size of the volume. The ratio of IOPS to allocated storage (in GiB) must be at least 0.5. Amazon RDS provides that IOPS rate for the DB instance until you change it.

The following table shows the range of Provisioned IOPS and storage size range for each database engine.

Database engine	Range of Provisioned IOPS	Range of storage
MariaDB	1,000–80,000 IOPS	100 GiB–64 TiB
SQL Server Enterprise, Standard, and Web Editions	1,000–64,000 IOPS*	20 GiB–16 TiB
SQL Server Express Edition	1,000–64,000 IOPS*	100 GiB–16 TiB
MySQL	1,000–80,000 IOPS	100 GiB–64 TiB
Oracle	1,000–80,000 IOPS	100 GiB–64 TiB
PostgreSQL	1,000–80,000 IOPS	100 GiB–64 TiB

Combining Provisioned IOPS storage with Multi-AZ deployments or read replicas

For production OLTP use cases, we recommend that you use Multi-AZ deployments for enhanced fault tolerance with Provisioned IOPS storage for fast and predictable performance.

You can also use Provisioned IOPS SSD storage with read replicas for MySQL, MariaDB or PostgreSQL. The type of storage for a read replica is independent of that on the primary DB instance. For example, you might use General Purpose SSD for read replicas with a primary DB instance that uses Provisioned IOPS SSD storage to reduce costs. However, your read replica's performance in this case might differ from that of a configuration where both the primary DB instance and the read replicas use Provisioned IOPS SSD storage.

Getting the best performance from Amazon RDS Provisioned IOPS SSD storage

If your workload is I/O constrained, using Provisioned IOPS SSD storage can increase the number of I/O requests that the system can process concurrently. Increased concurrency allows for decreased latency because I/O requests spend less time in a queue. Decreased latency allows for faster database commits, which improves response time and allows for higher database throughput.

Provisioned IOPS SSD storage provides a way to reserve I/O capacity by specifying IOPS. However, as with any other system capacity attribute, its maximum throughput under load is constrained by the resource that is consumed first. That resource might be network bandwidth, CPU, memory, or database internal resources.

Magnetic storage

Amazon RDS also supports magnetic storage for backward compatibility. We recommend that you use General Purpose SSD or Provisioned IOPS SSD for any new storage needs. The following are some limitations for magnetic storage:

- Doesn't allow you to scale storage when using the SQL Server database engine.
- Doesn't support storage autoscaling.
- Doesn't support elastic volumes.
- Limited to a maximum size of 3 TiB.
- Limited to a maximum of 1,000 IOPS.

Monitoring storage performance

Amazon RDS provides several metrics that you can use to determine how your DB instance is performing. You can view the metrics on the summary page for your instance in Amazon RDS Management Console. You can also use Amazon CloudWatch to monitor these metrics.

The following metrics are useful for monitoring storage for your DB instance:

- **IOPS** – The number of I/O operations completed each second. This metric is reported as the average IOPS for a given time interval. Amazon RDS reports read and write IOPS separately on 1-minute intervals. Total IOPS is the sum of the read and write IOPS. Typical values for IOPS range from zero to tens of thousands per second.
- **Latency** – The elapsed time between the submission of an I/O request and its completion. This metric is reported as the average latency for a given time interval. Amazon RDS reports read and write latency separately on 1-minute intervals in units of seconds. Typical values for latency are in the millisecond (ms). For example, Amazon RDS reports 2 ms as 0.002 seconds.

Monitoring storage performance

- **Throughput** – The number of bytes each second that are transferred to or from disk. This metric is reported as the average throughput for a given time interval. Amazon RDS reports read and write throughput separately on 1-minute intervals using units of megabytes per second (MB/s). Typical values for throughput range from zero to the I/O channel's maximum bandwidth.
- **Queue Depth** – The number of I/O requests in the queue waiting to be serviced. These are I/O requests that have been submitted by the application but have not been sent to the device because the device is busy servicing other I/O requests. Time spent waiting in the queue is a component of latency and service time (not available as a metric). This metric is reported as the average queue depth for a given time interval. Amazon RDS reports queue depth in 1-minute intervals. Typical values for queue depth range from zero to several hundred.

Measured IOPS values are independent of the size of the individual I/O operation. This means that when you measure I/O performance, you should look at the throughput of the instance, not simply the number of I/O operations.

Factors that affect storage performance

Both system activities and database workload can affect storage performance.

System activities

The following system-related activities consume I/O capacity and might reduce database instance performance while in progress:

- Multi-AZ standby creation
- Read replica creation
- Changing storage types

Factors that affect storage performance

Database workload

In some cases your database or application design results in concurrency issues, locking, or other forms of database contention. In these cases, you might not be able to use all the provisioned bandwidth directly. In addition, you may encounter the following workload-related situations:

- The throughput limit of the underlying instance type is reached.
- Queue depth is consistently less than 1 because your application is not driving enough I/O operations.
- You experience query contention in the database even though some I/O capacity is unused.

If there isn't at least one system resource that is at or near a limit, and adding threads doesn't increase the database transaction rate, the bottleneck is most likely contention in the database. The most common forms are row lock and index page lock contention, but there are many other possibilities. If this is your situation, you should seek the advice of a database performance tuning expert.

Factors that affect storage performance

DB instance class

To get the most performance out of your Amazon RDS database instance, choose a current generation instance type with enough bandwidth to support your storage type. For example, you can choose EBS-optimized instances and instances with 10-gigabit network connectivity.

We encourage you to use the latest generation of instances to get the best performance. Previous generation DB instances have a lower instance storage limit. The following table shows the maximum storage that each DB instance class can scale to for each database engine. All values are in tebibytes (TiB).

Factors that affect storage performance

Instance class	MariaDB	Microsoft SQL Server	MySQL	Oracle	PostgreSQL
db.m5 – Latest Generation Standard Instance Classes					
db.m5.24xlarge	64	16	64	64	64
db.m5.16xlarge	64	16	64	64	64
db.m5.12xlarge	64	16	64	64	64
db.m5.8xlarge	64	16	64	64	64
db.m5.4xlarge	64	16	64	64	64
db.m5.2xlarge	64	16	64	64	64
db.m5.xlarge	64	16	64	64	64
db.m5.large	64	16	64	64	64
db.m4 – Current Generation Standard Instance Classes					
db.m4.16xlarge	64	16	64	64	64
db.m4.10xlarge	64	16	64	64	64
db.m4.4xlarge	64	16	64	64	64
db.m4.2xlarge	64	16	64	64	64
db.m4.xlarge	64	16	64	64	64
db.m4.large	64	16	64	64	64

Instance class	MariaDB	Microsoft SQL Server	MySQL	Oracle	PostgreSQL
db.r5 – Latest Generation Memory Optimized Instance Classes					
db.r5.24xlarge	64	16	64	64	64
db.r5.16xlarge	64	16	64	64	64
db.r5.12xlarge	64	16	64	64	64
db.r5.8xlarge	64	16	64	64	64
db.r5.4xlarge	64	16	64	64	64
db.r5.2xlarge	64	16	64	64	64
db.r5.xlarge	64	16	64	64	64
db.r5.large	64	16	64	64	64
db.r4 – Current Generation Memory Optimized Instance Classes					
db.r4.16xlarge	64	16	64	64	64
db.r4.8xlarge	64	16	64	64	64
db.r4.4xlarge	64	16	64	64	64
db.r4.2xlarge	64	16	64	64	64
db.r4.xlarge	64	16	64	64	64
db.r4.large	64	16	64	64	64

Factors that affect storage performance

Instance class	MariaDB	Microsoft SQL Server	MySQL	Oracle	PostgreSQL
db.t3 – Latest Generation Burstable Performance Instance Classes					
db.t3.2xlarge	16	16	16	64	64
db.t3.xlarge	16	16	16	64	64
db.t3.large	16	16	16	64	64
db.t3.medium	16	16	16	32	32
db.t3.small	16	16	16	32	16
db.t3.micro	16	16	16	32	16

For Oracle, scaling up to 80,000 IOPS is only supported on the following instance classes.

- db.m5.24xlarge
- db.r5.24xlarge
- db.x1.32xlarge
- db.x1e.32xlarge



Creating a MySQL DB
instance and connecting to
a database on a MySQL DB
instance

Creating a MySQL DB instance: Console

You can create a DB instance running MySQL with the AWS Management Console with **Easy Create** enabled or disabled. With **Easy Create** enabled, you specify only the DB engine type, DB instance size, and DB instance identifier. **Easy Create** uses the default setting for other configuration options. With **Easy Create** not enabled, you specify more configuration options when you create a database, including ones for availability, security, backups, and maintenance.

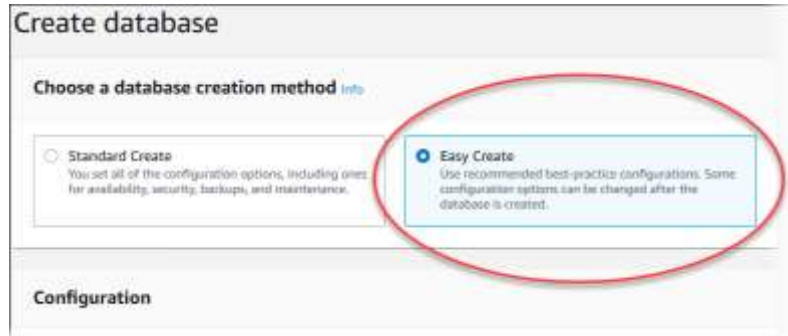
In this example, you use **Easy Create** to create a DB instance running the MySQL database engine with a db.t2.micro DB instance class.

To create a MySQL DB instance with Easy Create enabled

1. Sign in to the AWS Management Console and open the Amazon RDS console at <https://console.aws.amazon.com/rds/>.
2. In the upper-right corner of the Amazon RDS console, choose the AWS Region in which you want to create the DB instance.

Creating a MySQL DB instance: Console

3. In the navigation pane, choose **Databases**.
4. Choose **Create database** and make sure that **Easy Create** is chosen.



5. In **Configuration**, choose **MySQL**.
6. For **DB instance size**, choose **Free tier**.

Creating a MySQL DB instance: C

7. For **DB instance identifier**, enter a name for the DB instance, or leave the default name.
8. For **Master username**, enter a name for the master user, or leave the default name.

The **Create database** page should look similar to the following image.

9. To use an automatically generated master password for the DB instance, enable **Auto generate a password**.

To enter your master password, disable **Auto generate a password**, and then enter the same password in **Master password** and **Confirm password**.

The screenshot shows the 'Create database' page in the AWS Management Console. At the top, it says 'Choose a database creation method' with two options: 'Standard Create' (disabled) and 'Easy Create' (selected). Below this is the 'Configuration' section. Under 'Engine type', 'MySQL' is selected. Under 'DB instance size', 'Free tier' is selected. The 'DB instance identifier' field contains 'database-1'. The 'Master username' field contains 'admin'. At the bottom, the 'Auto generate a password' checkbox is checked.

Create database

Choose a database creation method

- ☐ Standard Create
- ☒ Easy Create

Configuration

Engine type

- ☐ Amazon Aurora
- ☒ MySQL
- ☐ MariaDB
- ☐ PostgreSQL
- ☐ Oracle
- ☐ Microsoft SQL Server

DB instance size

- ☐ Production
- ☐ Dev/Test
- ☒ Free tier

DB instance identifier

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in this current AWS Region.

database-1

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in 'mydbinstance'). Constraints: 1 to 63 alphanumeric characters or hyphens (1 to 15 for SQL Server). First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Master username

Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. First character must be a letter.

☒ Auto generate a password

Amazon RDS can generate a password for you, or you can specify your own password.

Creating a MySQL DB instance: Console

10. (Optional) Open **View default settings for Easy create**.

You can examine the default settings used when **Easy Create** is enabled. If you want to change one or more settings during database creation, choose **Standard Create** to set them. The **Editable after database creation** column shows which options you can change after database creation. To change a setting with **No** in that column, use **Standard Create**. For settings with **Yes** in that column, you can either use **Standard Create** or modify the DB instance after it is created to change the setting.

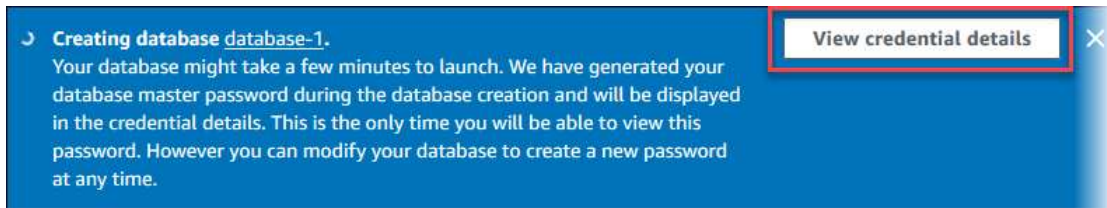
▼ View default settings for Easy create		
Easy create sets the following configurations to their default values, some of which can be changed later. If you want to change any of these settings now, use Standard Create .		
Configuration ▼	Value	Editable after database is created ▲
Encryption	Enabled	No
VPC	Default VPC (vpc-1234567a)	No
Option Group	default:oracle-se2-19	Yes
Subnet Group	default-vpc-1234567a	Yes
Automatic Backups	Enabled	Yes
VPC Security Group	sg-1a2bcd3e	Yes
Publicly Accessible	No	Yes
Database Port	1521	Yes

Creating a MySQL DB instance: Console

11. Choose **Create database**.

If you chose to use an automatically generated password, the **View credential details** button appears on the **Databases** page.

To view the master username and password for the DB instance, choose **View credential details**.

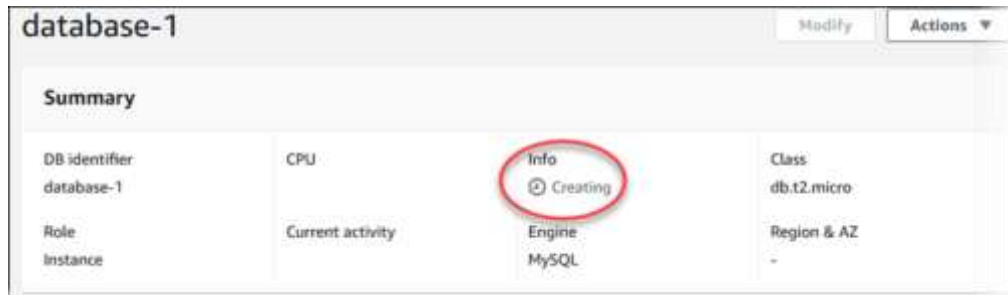


You can use the username and password that appears to connect to the DB instance as the master user.

Creating a MySQL DB instance: Console

12. In the **Databases** list, choose the name of the new MySQL DB instance.

On the RDS console, the details for new DB instance appear. The DB instance has a status of **creating** until the DB instance is ready to use. When the state changes to **available**, you can connect to the DB instance. Depending on the DB instance class and the amount of storage, it can take up to 20 minutes before the new instance is available.

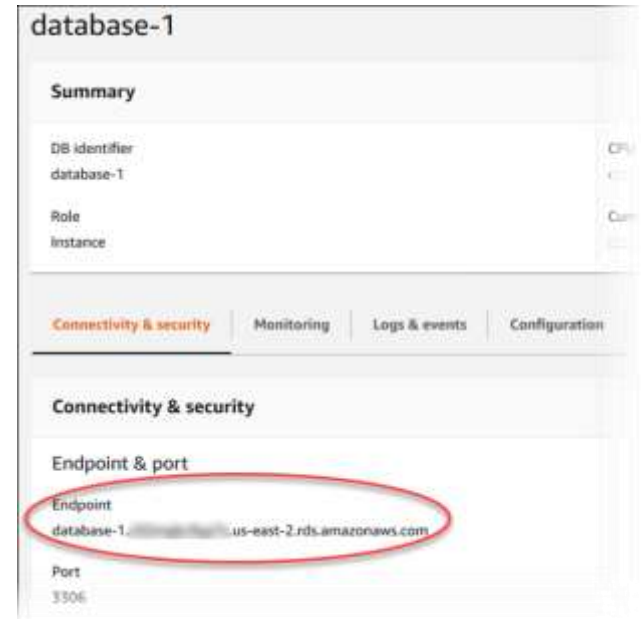


Connecting to a database on a DB instance running the MySQL database engine

After Amazon RDS provisions your DB instance, you can use any standard SQL client application to connect to a database on the DB instance. In this example, you connect to a database on a MySQL DB instance using MySQL monitor commands. One GUI-based application you can use to connect is MySQL Workbench.

To connect to a database on a DB instance using MySQL monitor

1. Find the endpoint (DNS name) and port number for your DB instance.
 - a. Open the RDS console and then choose **Databases** to display a list of your DB instances.
 - b. Choose the MySQL DB instance name to display its details.
 - c. On the **Connectivity & security** tab, copy the endpoint. Also, note the port number. You need both the endpoint and the port number to connect to the DB instance.



Connecting to a database on a DB instance running the MySQL database engine

2. Download a SQL client that you can use to connect to the DB instance.
You can connect to an Amazon RDS MySQL DB instance by using tools like the MySQL command line utility. One GUI-based application you can use to connect is MySQL Workbench.
3. Connect to the a database on a MySQL DB instance. For example, enter the following command at a command prompt on a client computer to connect to a database on a MySQL DB instance using the MySQL client. Substitute the DNS name for your DB instance for `<endpoint>`, the master user name you used for `<mymasteruser>`, and provide the master password you used when prompted for a password.

```
PROMPT> mysql -h <endpoint> -P 3306 -u <mymasteruser> -p
```

Connecting to a database on a DB instance running the MySQL database engine

After you enter the password for the user, you should see output similar to the following.

```
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 350  
Server version: 5.6.40-log MySQL Community Server (GPL)  
  
Type 'help;' or '\h' for help. Type '\c' to clear the buffer.  
  
mysql>
```

If you can't connect to your MySQL DB instance, two common causes of connection failures to a new DB instance are:

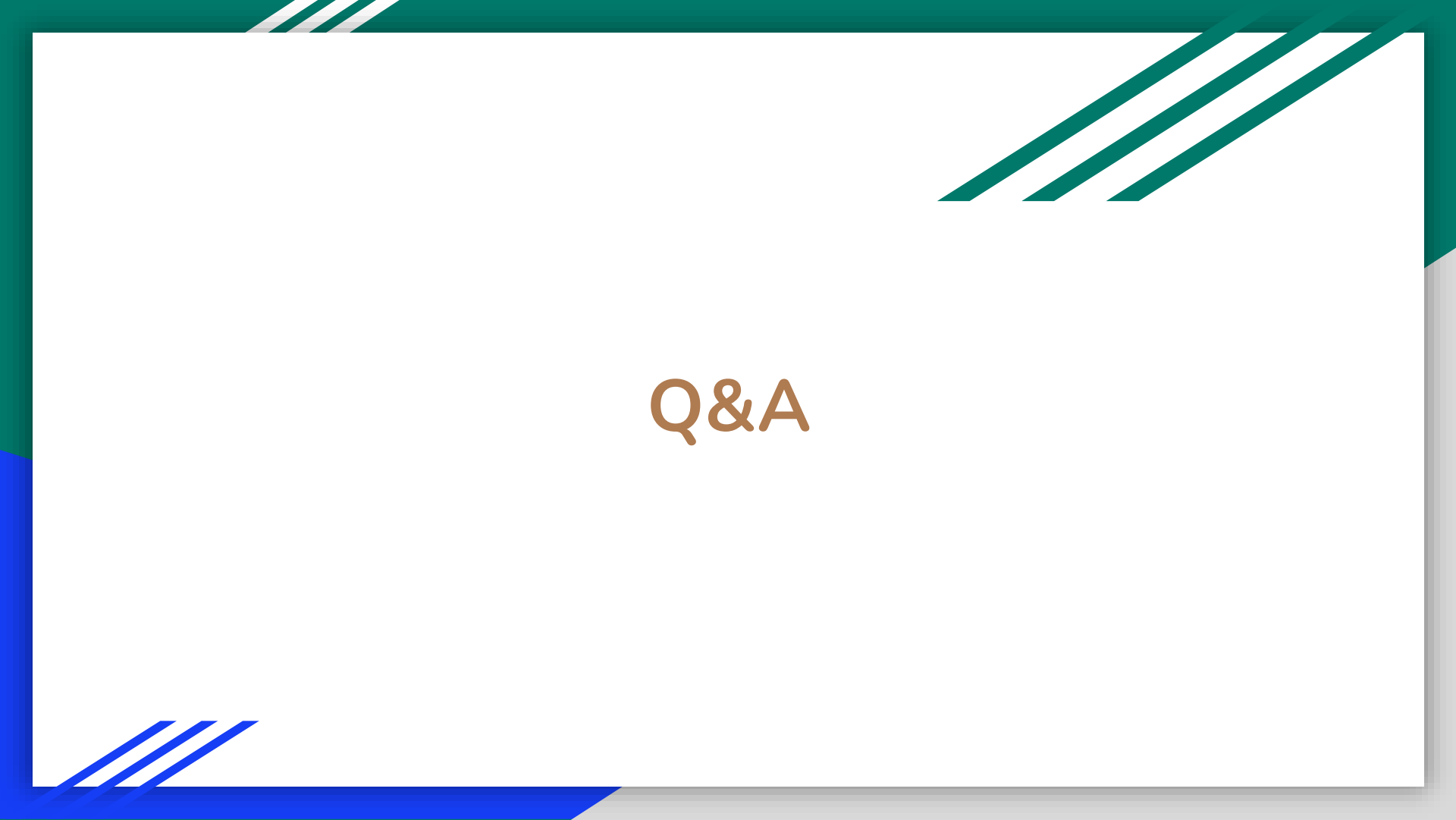
- The DB instance was created using a security group that does not authorize connections from the device or Amazon EC2 instance where the MySQL application or utility is running. If the DB instance was created in a VPC, it must have a VPC security group that authorizes the connections. If the DB instance was created outside of a VPC, it must have a DB security group that authorizes the connections.
- The DB instance was created using the default port of 3306, and your company has firewall rules blocking connections to that port from devices in your company network. To fix this failure, recreate the instance with a different port.

Deleting a DB instance

After you have connected to the sample DB instance that you created, you should delete the DB instance so you are no longer charged for it.

To delete a DB instance with no final DB snapshot

1. Sign in to the AWS Management Console and open the Amazon RDS console at <https://console.aws.amazon.com/rds/>.
2. In the navigation pane, choose **Databases**.
3. Choose the DB instance that you want to delete.
4. For **Actions**, choose **Delete**.
5. For **Create final snapshot?**, choose **No**, and select the acknowledgment.
6. Choose **Delete**.



Q&A